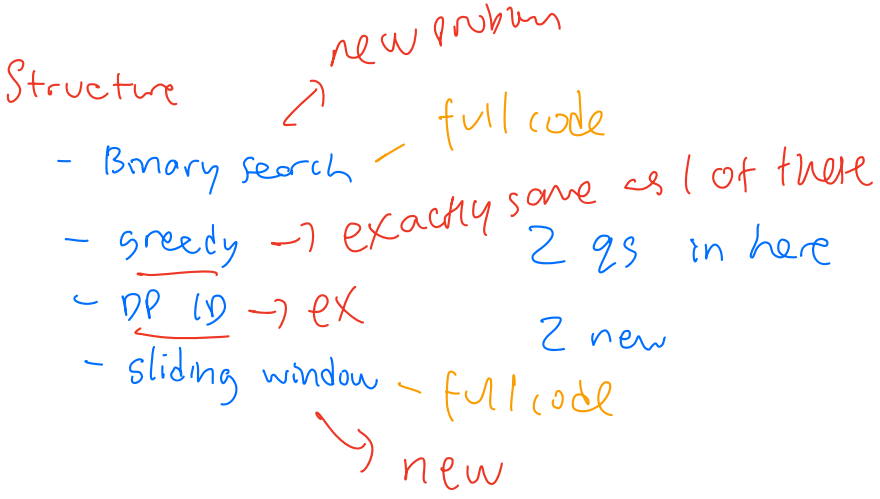
Midterm review



House robber



```
Input: money = [8, 9, 8, 3]
Output: 16
```

Formalized

Input: `money`, an array of non-negative integers -- the cash in each house along the street.

Output: the max total you can rob, without robbing two **adjacent** houses.

- DP int[] houses
- dp[i] = best score if houses[0...i]
- recurrence dp[i] = max(
  - rob house at i → houses[i] + dp[i-2]
  - not rob house at i → dp[i-1])
- Runtime: O(n)
  - why: check each house once

```
for i -> 0 -> n-1
    dp[i] = ...        } O(1)
```

Find peak in mountain

```
Input: arr = [1, 3, 5, 4, 2]
Output: 2
```

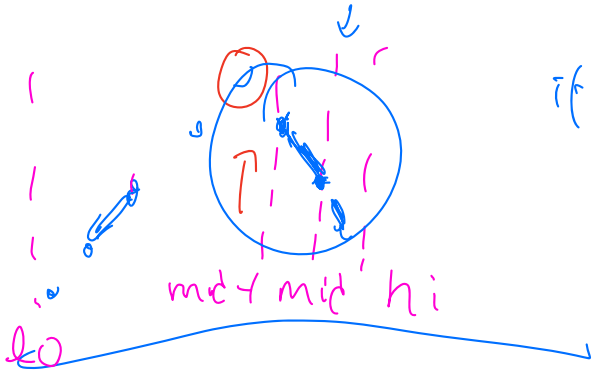


Formalized

Input: `arr`, a mountain array: its values strictly increase to a single peak, then strictly decrease.

Output: the index of the peak, in O(log n) time.

- Binary search
- check if arr[i] is peak ?
  - ↳ compare prev & next.



if arr[i] > prev & arr[i] < next

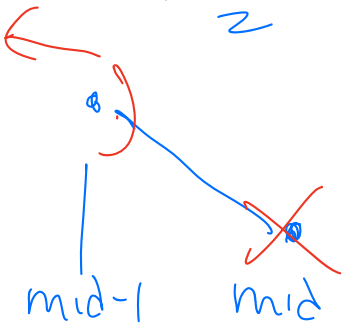
→ prev / arr[i] / next

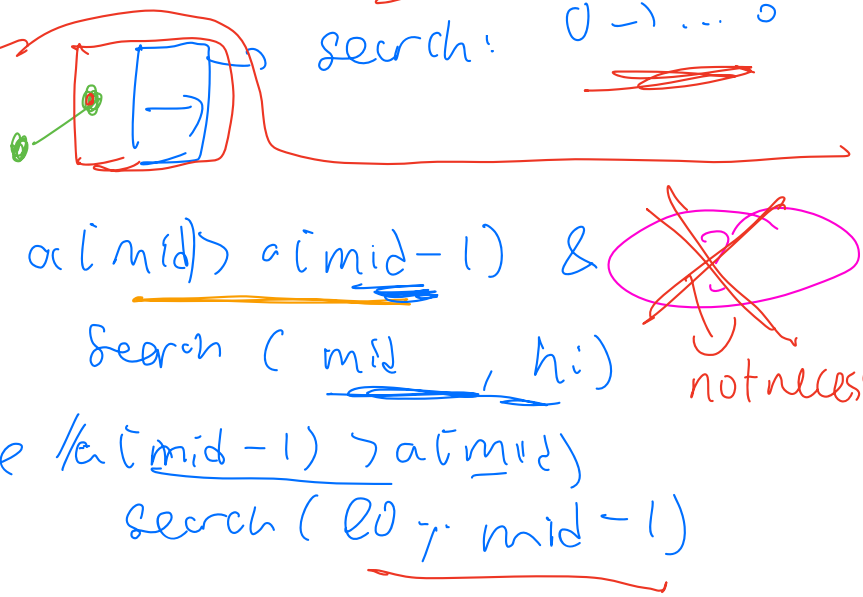
search: 0 → ... ?

lo = 0

hi = n-1

mid = (lo+hi)/2





if a[mid] > a[mid-1] &

search (mid, hi)

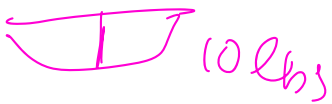
not necessary

else if a[mid-1] > a[mid]

search (lo, mid-1)

Boats to save people

```
Input: people = [8, 7, 6, 4, 3, 3, 2, 1], limit = 10
Output: 4
```



Formalized

Input: `people`, an array of weights, and `limit`, the weight capacity of one boat.

Output: the minimum number of boats, given each boat carries at most 2 people and at most `limit` total weight.

greedy.

01. pair heaviest to lightest
   - if htl > limit, just put heaviest
   - if h > limit ⇒ exclude R.i.p :(

02. heaviest + next heaviest that still fits
   → O(log n) w/ AVL trees

Implementation

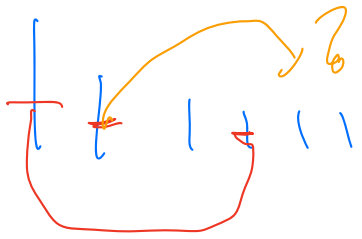
02. sort. → O(n log n)
   for each heaviest

binary search ← find next heaviest that fits
log n

O(n) if copy :) ← ship them.

Guess: n log n. intuition of less & less work

Why hard: how to keep track of who's shipped?

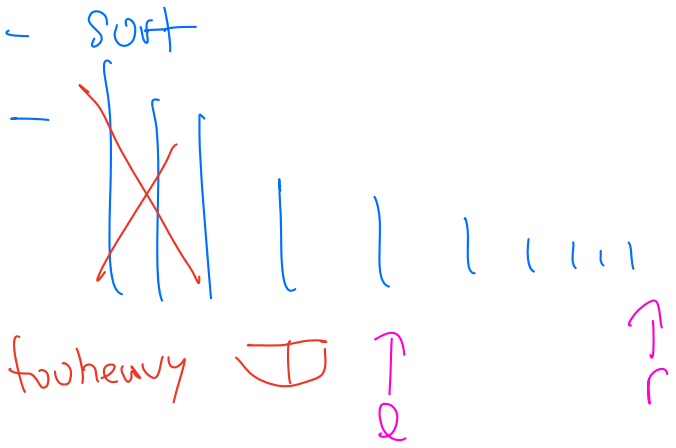


boolean[] used..

want:
data structure
remove(item)
find first G> (item)
} log(n)

Balanced trees

01_implement: par. heavier w/ constant




```
while(l<=r)
  if a[l]+a[r]<=l
    l++
    r--
    countboat++
  else
    l++
```

runtime:

sort → O(nlogn)

two-pointer → O(n)

Lemonade stand streak

You ran a lemonade stand for 10 days. Your daily profit, in dollars:

```
Input: profit = [40, -15, 25, -30, -10, 55, 20, -5, 35, -20]
       K = 2
Output: 5
```

losses in consecutive period.

profitable > 40% of the days

Q: longest period at most 2 loss days?

Formalized

Input: `profit`, an array of integers -- one day's profit each, negative on a losing day -- and an integer K.

Output: the length of the longest **contiguous** stretch of days containing at most K losing days.

DP

dp[i][j] = len of longest ends at i & has at most j losses

Sliding window.

l, r = 0 badDays = 0

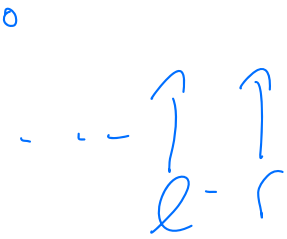
max = 0

for r : 0 → n-1

insert head:
- if arr[r] < 0
  - badDays ++

move the tail:
- while badDays > limit
  - if arr[l] < 0
    - badDays --
  - l ++
- max = max(r - l + 1, max)

return max



Max subarray sum

```
Input: nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Output: 6
```

dp[i] = max sum if have to end at exactly i

recurrence. — include previous → nums[i] + dp[i-1]

max(

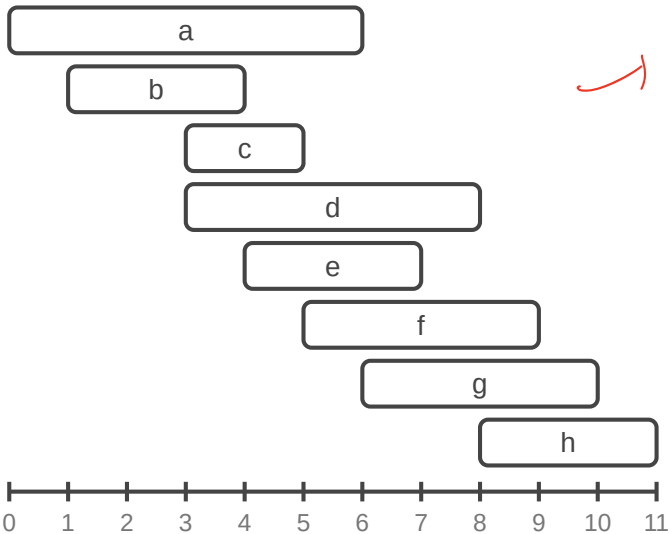
— by itself: → nums[i])

Formalized

Input: `nums`, an array of integers.

Output: the largest sum of a **contiguous**, non-empty subarray.

Interval scheduling



→ lecture greedy. sort by asc end time

Output: 3

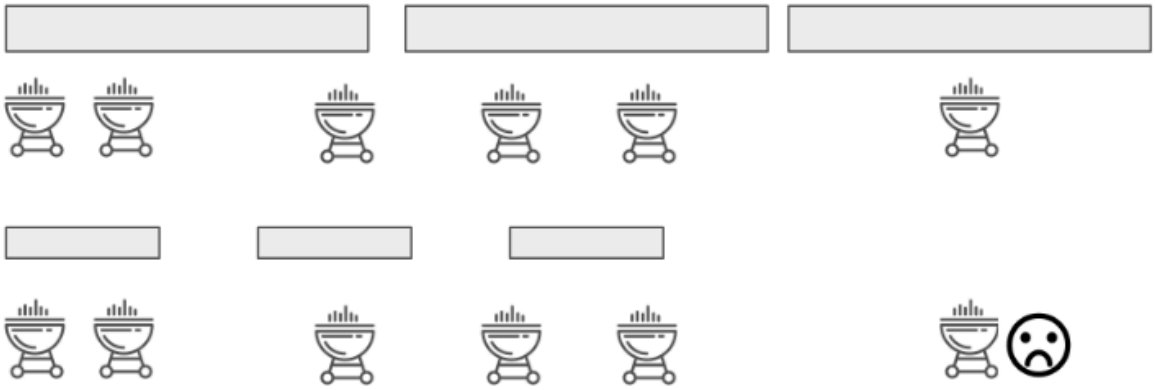
Formalized

Input: `intervals`, a list of `(start, end)` pairs.

Output: the max number of intervals that pairwise do not overlap.

Umbrellas for the BBQ

Skip



```
Input: friends = [1, 2, 4, 7, 8, 12], umbrellas = 3
Output: 3
```

Formalized

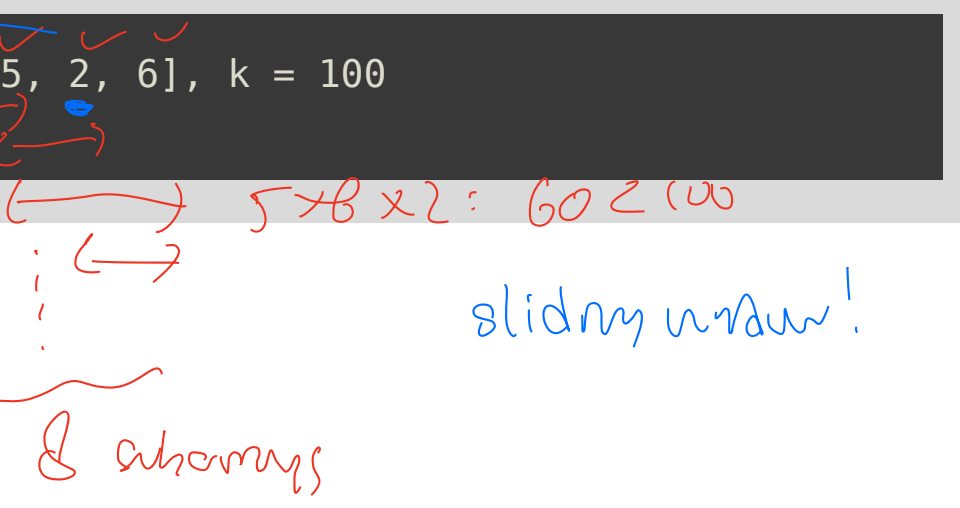
Input: `friends`, the positions of your friends on a line, and `k`, the number of umbrellas. An umbrella of length `L` placed at `x` covers `[x, x + L]`.

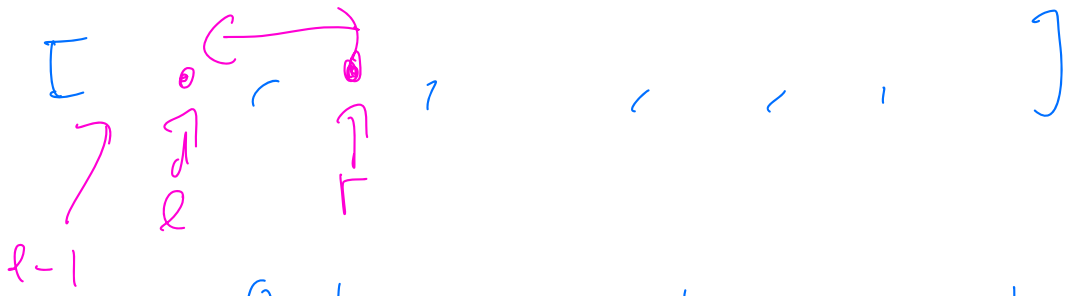
Output: the minimum length `L` such that `k` umbrellas of length `L` cover every friend.

tricky, midterm is easier

Subarray product less than k

```
Input: nums = [10, 5, 2, 6], k = 100
Output: 8
```





Q: how many subarrays end at r & mut ≤ k?

A: sliding window find left most pairm

cnst: r - l + 1

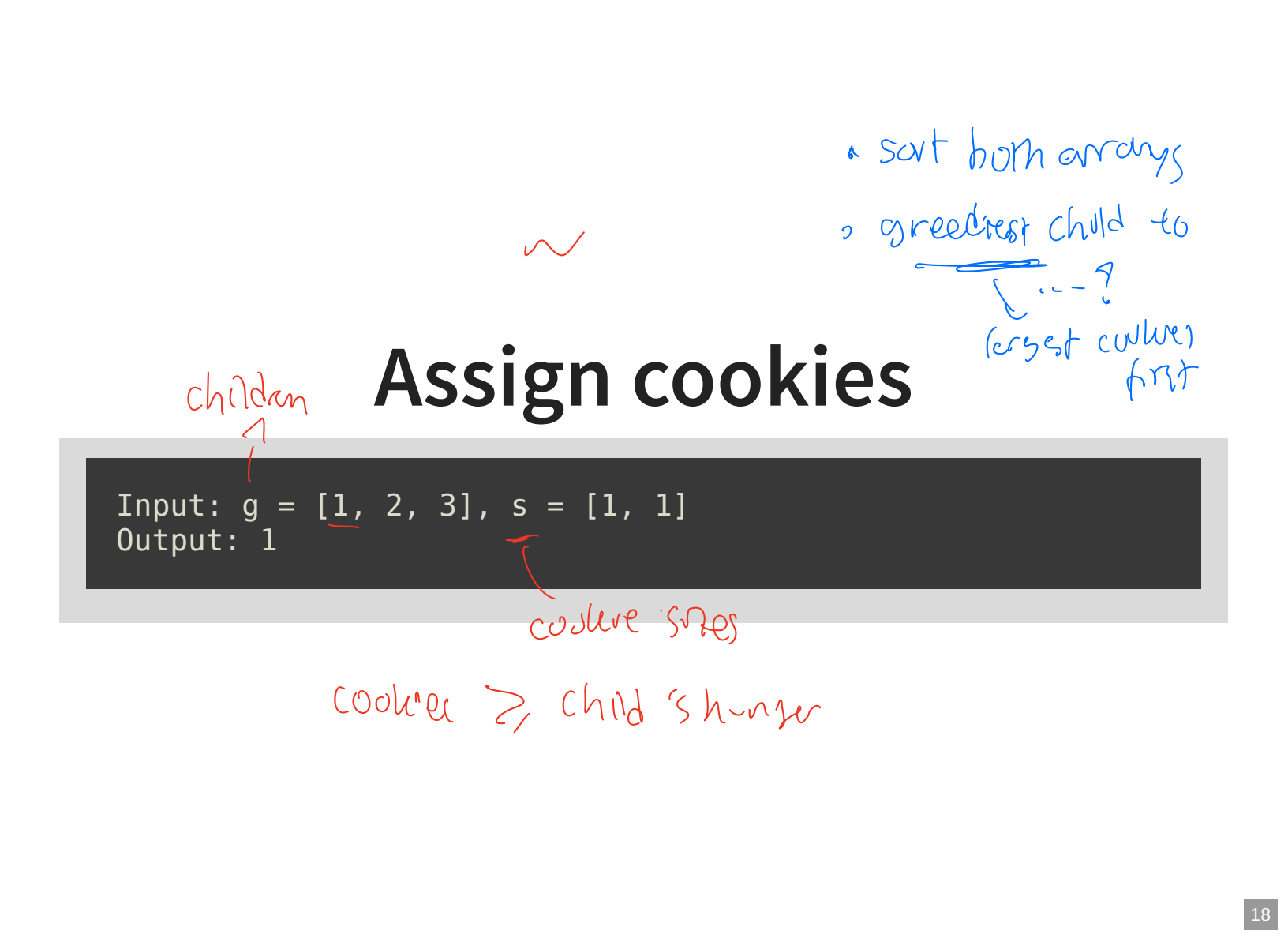
Formalized

Input: `nums`, an array of **positive** integers, and an integer `k`.

Output: the number of **contiguous** subarrays whose product is strictly less than `k`.

Assign cookies

```
Input: g = [1, 2, 3], s = [1, 1]
Output: 1
```



Formalized

Input: `g`, the children's greed factors, and `s`, the cookie sizes. Cookie `j` satisfies child `i` if `s[j] >= g[i]`, and each child gets at most one cookie.

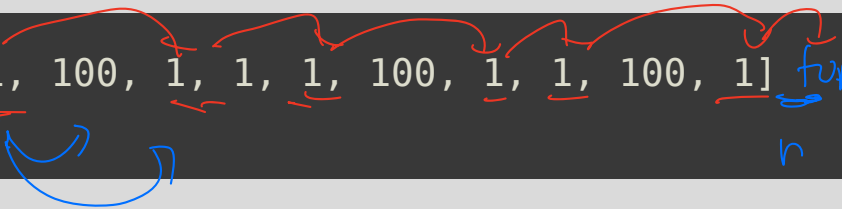
Output: the maximum number of satisfied children.

dp[i]: min cost it can start to reach the end at i or it!

dp[i] = min cost if can start at 0/1, and reach i

Min cost climbing stairs

```
Input: cost = [1, 100, 1, 1, 1, 100, 1, 1, 100, 1]
Output: 6
```



Formalized

Input: `cost`, an array of positive integers -- the cost of stepping on each stair. After paying, you climb one or two stairs. You may start on stair `0` or `1` for free.

Output: the minimum total cost to reach the top, one past the last stair.